

```
In [3]: import pandas as pd

# Load the data you saved in the previous step
X_train = pd.read_csv('X_train_processed.csv')
X_test = pd.read_csv('X_test_processed.csv')
y_train = pd.read_csv('y_train_processed.csv').values.ravel() # Con
y_test = pd.read_csv('y_test_processed.csv').values.ravel()

print("Data loaded! Ready for modeling.")
print(f"Training shape: {X_train.shape}")
```

Data loaded! Ready for modeling.
Training shape: (242, 22)

```
In [4]: from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report,

# 1. Initialize the model
baseline_model = LogisticRegression(max_iter=1000)

# 2. Train the model
baseline_model.fit(X_train, y_train)

# 3. Make predictions on the test set
y_pred_baseline = baseline_model.predict(X_test)

# 4. Show the results (Evaluation)
print("--- Baseline: Logistic Regression ---")
print(f"Accuracy: {accuracy_score(y_test, y_pred_baseline):.2f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred_baseline))
```

--- Baseline: Logistic Regression ---
Accuracy: 0.51

Classification Report:

	precision	recall	f1-score	support
0	0.79	0.90	0.84	29
1	0.17	0.17	0.17	12
2	0.22	0.22	0.22	9
3	0.14	0.14	0.14	7
4	0.00	0.00	0.00	4
accuracy			0.51	61
macro avg	0.26	0.29	0.27	61
weighted avg	0.46	0.51	0.48	61

```
/Users/alikhansagat/Library/Python/3.9/lib/python/site-packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.
```

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(
/Users/alikhansagat/Library/Python/3.9/lib/python/site-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/Users/alikhansagat/Library/Python/3.9/lib/python/site-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/Users/alikhansagat/Library/Python/3.9/lib/python/site-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

- Accuracy: The percentage of patients the model guessed correctly.
- Classification Report: This shows how well the model avoids "False Positives" (saying someone has disease when they don't) and "False Negatives" (missing a real case of disease)

```
In [5]: from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report

# 1. Инициализируем Дерево решений
tree_model = DecisionTreeClassifier(random_state=42)

# 2. Обучаем модель на тех же данных
tree_model.fit(X_train, y_train)

# 3. Делаем предсказание
y_pred_tree = tree_model.predict(X_test)

# 4. Выводим результаты
print("---- Model 2: Decision Tree ----")
print(f"Accuracy: {accuracy_score(y_test, y_pred_tree):.2f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred_tree))
```

--- Model 2: Decision Tree ---

Accuracy: 0.54

Classification Report:

	precision	recall	f1-score	support
0	0.79	0.76	0.77	29
1	0.29	0.33	0.31	12
2	0.40	0.44	0.42	9
3	0.29	0.29	0.29	7
4	0.50	0.25	0.33	4
accuracy			0.54	61
macro avg	0.45	0.41	0.42	61
weighted avg	0.55	0.54	0.54	61

```
In [6]: from sklearn.neighbors import KNeighborsClassifier

# 1. Инициализируем модель KNN (возьмем 5 соседей для начала)
knn_model = KNeighborsClassifier(n_neighbors=5)

# 2. Обучаем модель
knn_model.fit(X_train, y_train)

# 3. Делаем предсказание
y_pred_knn = knn_model.predict(X_test)

# 4. Выводим результаты
print("--- Model 3: KNN ---")
print(f"Accuracy: {accuracy_score(y_test, y_pred_knn):.2f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred_knn))
```

--- Model 3: KNN ---

Accuracy: 0.44

Classification Report:

	precision	recall	f1-score	support
0	0.55	0.90	0.68	29
1	0.10	0.08	0.09	12
2	0.00	0.00	0.00	9
3	0.00	0.00	0.00	7
4	0.00	0.00	0.00	4
accuracy			0.44	61
macro avg	0.13	0.20	0.16	61
weighted avg	0.28	0.44	0.34	61

```

/Users/alikhansagat/Library/Python/3.9/lib/python/site-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/Users/alikhansagat/Library/Python/3.9/lib/python/site-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/Users/alikhansagat/Library/Python/3.9/lib/python/site-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))

```

In [7]: **from** sklearn.svm **import** SVC

```

# Создаем модель SVM. Параметр C=10 и gamma='scale' обычно помогают
svm_model = SVC(kernel='rbf', C=10, gamma='scale', random_state=42)

# Обучаем
svm_model.fit(X_train, y_train)

# Проверяем
y_pred_svm = svm_model.predict(X_test)

print("---- Model 4: SVM (RBF Kernel) ----")
print(f"Accuracy: {accuracy_score(y_test, y_pred_svm):.2f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred_svm))

```

--- Model 4: SVM (RBF Kernel) ---

Accuracy: 0.48

Classification Report:

	precision	recall	f1-score	support
0	0.48	1.00	0.64	29
1	0.00	0.00	0.00	12
2	0.00	0.00	0.00	9
3	0.00	0.00	0.00	7
4	0.00	0.00	0.00	4
accuracy			0.48	61
macro avg	0.10	0.20	0.13	61
weighted avg	0.23	0.48	0.31	61

```
/Users/alikhansagat/Library/Python/3.9/lib/python/site-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/Users/alikhansagat/Library/Python/3.9/lib/python/site-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/Users/alikhansagat/Library/Python/3.9/lib/python/site-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

In []:

In []:

In []: